A Multi-Objective Metaheuristic Approach for Student–Industry Placement Allocation with Academic Supervision

Juan Miguel Alberola Oltra, José Antonio Rodríguez Moreno, Víctor Sánchez Anguix

Abstract. Student–Industry Placement Allocation (SIPA) consists of assigning students to internship placements offered by companies. Existing competence-based formulations consider student and company utilities, competence matching, distances, quotas, and placement dependencies, but they do not explicitly incorporate the academic supervision layer. This may lead to assignments that are feasible from the student–company perspective but difficult to manage in practice.

This paper extends the SIPA problem by incorporating academic supervisors, educational centers, supervision capacities, geographical dispersion, and workload balance among professors. To address the resulting multi-objective problem, we propose HH-LAP-SIPA, a hybrid metaheuristic that combines feasible placement selection, placement–professor assignment, and exact Linear Assignment Problems for the student–placement layer, while maintaining an external Pareto archive.

The approach is evaluated on 20 synthetic instances ranging from 200 to 1000 students and compared with random, NSGA-III, and GRASP baselines. The experimental results show that HH-LAP-SIPA obtains higher-quality Pareto fronts in terms of hypervolume, with the lightweight variant achieving the best overall performance.

Keywords: Student–Industry Placement Allocation; academic supervision; multi-objective optimization; metaheuristics; Pareto optimization; hypervolume.

1 Introduction

The allocation of students to industry placements is a relevant task in many educational contexts, especially in training programs where professional experience is part of the learning process. This process, known as *Student–Industry Placement Allocation* (SIPA), consists of assigning students to placement positions offered by companies.

This work takes as its starting point the model proposed in *An Integer Linear Programming Model Based on Competences for Student–Industry Placement Allocation* [1]. That model formulates the SIPA problem by mainly considering the relationship between students and companies, and incorporates elements such as the preferences of both parties, competence-based matching between students and placements, the student–company distance, company quotas, and possible interdependencies between placements. However, this

formulation does not explicitly incorporate the academic supervision layer, even though, in real internship allocation processes, professors are responsible for monitoring and evaluating students during their stay in the company.

By not explicitly considering academic supervision, an assignment that is valid from the student–company perspective may require additional organizational decisions before it can be implemented in a real setting. In particular, once students have been assigned to placements, those placements would still need to be assigned to academic supervisors, taking into account their supervision capacity and the location of the companies. If this dimension is not integrated into the model, the final solution may be difficult to manage from an academic perspective, for instance due to an unbalanced distribution of teaching workload or a high geographical dispersion of the placements assigned to the same supervisor. For this reason, this work addresses a modified version of the SIPA problem in which academic supervisors are explicitly incorporated as a new agent in the model.

The incorporation of professors introduces new organizational criteria that must be considered together with student and company satisfaction. In particular, the extended model considers the maximum supervision capacity of each professor, the effort associated with the geographical dispersion of supervised placements, and the workload balance among professors belonging to the same educational center. As a consequence, the resulting problem has a clearly multi-objective nature, since it combines utility objectives for students and companies with objectives related to the organizational feasibility of academic supervision.

These additional supervision-related criteria increase the complexity of the problem and reinforce the need to use approximate methods capable of obtaining high-quality solutions within reasonable computational times. In this context, this work proposes HH-LAP-SIPA, a multi-objective metaheuristic approach designed to solve the modified SIPA problem by simultaneously considering student and company utilities, as well as organizational criteria associated with academic supervision. The aim is not to obtain a single optimal assignment, but rather to explore different compromise solutions among the objectives of the problem.

The main contributions of this work are threefold. First, it formulates a modified version of the SIPA problem that incorporates academic supervision, supervision capacity, geographical dispersion, and workload balance. Second, it proposes HH-LAP-SIPA, a multi-objective metaheuristic approach to address the resulting problem. Third, it presents an experimental analysis against different reference algorithms, using hypervolume as the main multi-objective quality indicator.

2 Related Work

This section reviews the most relevant previous works to contextualize the proposed approach. First, educational assignment problems that share structural similarities with SIPA are presented, including the allocation of students to projects, groups, or supervisors. Next, works specifically addressing *Student–Industry Placement Allocation* are reviewed, with particular attention given to the competence-based model that serves as the starting point of this work. Finally, the scalability limitations associated with exact approaches are discussed, motivating the use of approximate methods to address larger instances and more complex versions of the problem.

2.1 Educational assignment problems

Assignment problems in educational contexts constitute a broad family of combinatorial problems in which students must be assigned to resources, activities, projects, groups, or

supervisors while satisfying different constraints. Although these problems are not equivalent to *Student–Industry Placement Allocation*, they share relevant structural elements with it, such as the presence of students, preferences, capacities, feasibility constraints, and, in some cases, academic supervisors.

A related problem is *Student–Project Allocation* (SPA), where students must be assigned to projects while taking into account their preferences and different constraints. Several works have addressed this problem through integer programming models or approaches focused on stability, preferences, and side constraints [2, 3]. These works are relevant because they share with SIPA the need to combine preferences, capacities, and feasibility constraints, although in SPA the assigned resource is usually an academic project rather than an industry placement offered by a company.

Another group of related works focuses on the formation of student teams. In these problems, the goal is not to assign students to external entities, but to divide a class into teams satisfying certain pedagogical or organizational criteria. Recently, an evolutionary metaheuristic has been proposed for forming teams in the classroom with constraints, considering flexible team sizes and compulsory or forbidden combinations of students [4]. Although this problem also differs from SIPA, it is relevant because it illustrates the importance of considering realistic constraints in educational assignment problems and the usefulness of metaheuristic methods when combinatorial complexity limits the applicability of exact approaches.

There are also works specifically focused on assigning students to academic supervisors. These problems consider aspects such as student and supervisor preferences, minimum and maximum supervision quotas, and workload balance among supervisors [5]. These elements are particularly close to the extension proposed in this work, since they explicitly introduce the figure of the academic supervisor. However, such approaches usually treat student–supervisor allocation as an independent problem, whereas in this work academic supervision is integrated into a broader SIPA problem that also considers students, companies, industry placements, and organizational criteria associated with supervision.

2.2 Student–Industry Placement Allocation

The *Student–Industry Placement Allocation* problem focuses on assigning students to internship placements offered by companies. Unlike other educational assignment problems, SIPA explicitly involves entities external to the educational institution, since the available placements depend on collaborating companies and on the conditions associated with each offer. Therefore, in addition to assigning each student to a suitable placement, the problem must consider constraints specific to the industry context, such as the number of available placements, the quotas agreed with each company, or the existence of placements that are related to each other.

Previous works have addressed the problem of assigning students to industry placements from specific perspectives. For example, a greedy algorithm has been proposed for a real case of student allocation in internship programs, considering the distance between the assigned professors and the locations of the internships [6]. Student placement allocation has also been studied using integer programming and *ant colony optimisation*, considering scenarios in which companies may offer several placements and express preferences over students [7]. These works show the practical relevance of the problem and the need to use optimization techniques, although they address only some of the criteria involved in real allocation scenarios.

A more directly related advance is the model proposed in *An Integer Linear Programming*

Model Based on Competences for Student–Industry Placement Allocation [1]. This model formulates SIPA as a multi-objective integer linear programming problem in which both student and company utilities are considered. Its main contribution is the representation of preferences through competence vectors, relating the students' current competences, the competences they wish to develop during the placement, and the competences required by the offered placements. In addition, it incorporates the student–company distance, minimum and maximum quotas per company, and interdependencies between placements.

This competence-based model constitutes the direct starting point of the present work. However, its formulation focuses on the student–company assignment and does not explicitly incorporate academic supervision as a structural part of the problem. Therefore, the proposed extension preserves the main elements of the competence-based model, but introduces academic supervisors and organizational criteria associated with supervision, such as supervision capacity, geographical dispersion, and workload balance.

2.3 Scalability limitations and approximate methods

Exact models based on integer linear programming provide a rigorous way to formulate the SIPA problem and incorporate relevant constraints from the allocation context, although their practical applicability may be limited as instance size increases. In the competence-based SIPA model, the number of variables and constraints grows with the number of students, placements, and companies, as well as with the presence of quotas and interdependencies between placements. As a consequence, exact resolution may require high computational times, especially in scenarios with a larger number of students or with more available placements than students.

This limitation has been experimentally analysed in previous work on the SIPA ILP model, where the resolution time was observed to increase significantly as the number of students and the number of available placements grew [8]. Moreover, although commercial solvers may provide better performance than non-commercial alternatives, they are not always available to institutions that require open and accessible solutions. These results show that, although exact approaches are suitable for formally defining and validating the problem, they may not be sufficient to address larger instances within reasonable computational times.

The need for approximate methods becomes even clearer in the modified version of the problem considered in this work. The incorporation of academic supervisors adds new decisions and organizational criteria, such as assigning placements to professors, supervision capacities, geographical dispersion, and workload balance. These elements increase the search space and make the exact resolution of the complete problem even more difficult. Therefore, it is necessary to study metaheuristic methods capable of generating feasible and high-quality solutions within reasonable computational times, especially in scenarios where different trade-offs among objectives must be explored.

3 Problem Formulation

The problem addressed in this work is formulated in this section. First, the original competence-based SIPA model, which constitutes the starting point of the proposal, is summarized. Then, the extension introduced through the incorporation of academic supervisors, supervision capacities, and organizational criteria related to geographical dispersion and workload balance is described. Finally, the constraints of the modified problem and its multi-objective formulation are presented.

3.1 Original SIPA model

The original competence-based SIPA model constitutes the starting point of the formulation proposed in this work [1]. In this model, an educational institution must assign a set of students to a set of internship placements offered by companies. Let S denote the set of students, P the set of available placements, and B the set of companies offering those placements. Each placement $j \in P$ belongs to a company $k \in B$, and $P_k \subseteq P$ denotes the set of placements offered by company k.

Each company may agree with the educational institution on a minimum quota q_k and a maximum quota Q_k of assigned students. Moreover, some placements may present interdependencies, for instance when several placements are part of the same company project and must be filled jointly. To represent this situation, a set $\mathcal{I}$ of interdependent placement groups is considered, where each group $I \in \mathcal{I}$ consists of a subset of placements that must be activated jointly.

The model also incorporates competence-based information. Let C denote the set of competences relevant to the allocation process, with $c = |C|$. Each placement $j \in P$ is associated with a competence relevance vector

$$\Phi_j = \langle \phi_{j1}, \ldots, \phi_{jc} \rangle,$$

where ϕ_{jl} represents the importance of competence $l \in C$ for carrying out placement j. Similarly, each student $i \in S$ is associated with two vectors: a current competence vector

$$\Psi_i = \langle \psi_{i1}, \ldots, \psi_{ic} \rangle,$$

where ψ_{il} represents the current level of student i in competence l, and a desired competence vector

$$\Theta_i = \langle \theta_{i1}, \ldots, \theta_{ic} \rangle,$$

where θ_{il} represents the interest of student i in developing competence l during the placement.

All competence vectors are assumed to be normalized so that their components belong to the interval $[0, 1]$. In addition, placement competence relevance vectors are assumed to be normalized consistently, so that the competence matching terms $\Phi_j \cdot \Theta_i$ and $\Phi_j \cdot \Psi_i$ also lie in $[0, 1]$.

Based on this information, the model defines two main utilities. On the one hand, the utility that placement j provides to student i, denoted by τ_{ij}, combines the student–company distance with the match between the competences that the student wishes to develop and the competences associated with the placement. In the original model, this utility is defined as

$$\tau_{ij} = w_i^d(1 - d_{ij}) + w_i^c(\Phi_j \cdot \Theta_i),$$

where d_{ij} represents the normalized distance between student i and the location of placement j, while w_i^d and w_i^c represent the relative importance assigned by student i to distance and competence matching, respectively, with

$$w_i^d + w_i^c = 1, \qquad w_i^d, w_i^c \in [0, 1].$$

On the other hand, the utility that student i provides to placement j, denoted by η_{ij}, measures the match between the student's current competences and the competences required by the placement:

$$\eta_{ij} = \Phi_j \cdot \Psi_i.$$

The basic decision variable of the original model is

$$x_{ij} = \begin{cases} 1, & \text{if student } i \text{ is assigned to placement } j, \\ 0, & \text{otherwise.} \end{cases}$$

Using this variable, the model aims to simultaneously maximize the aggregate utility of students and the aggregate utility of companies, ensuring that all students are assigned, that no placement is occupied by more than one student, and that placement interdependencies and the minimum and maximum company quotas are respected.

3.2 Extension with academic supervisors

The proposed modification incorporates academic supervisors into the SIPA formulation in order to capture organizational and logistical aspects that are not represented when only students and companies are considered. In real internship allocation processes, academic supervisors are responsible for monitoring and evaluating students, and therefore the feasibility of an allocation also depends on how supervision tasks are distributed.

In the modified model, professors are introduced as a third agent, and their inclusion affects several components of the original SIPA formulation. Specifically, the set of represented agents is extended, supervision-related distance and dispersion costs are introduced, supervision capacity constraints are added, and the objective structure is expanded to include academic and organizational criteria. Thus, the proposed formulation should be regarded as a structural modification of the original SIPA model.

The original SIPA model components are assumed to remain valid unless explicitly modified. The extension presented below introduces the additional sets, parameters, decision variable, and derived quantities required to model academic supervision. The corresponding constraints and objective components are presented in the following subsections.

Let T denote the set of professors, and let $\mathcal{E}$ denote the set of educational centers. Moreover, each professor $t \in T$ belongs to exactly one educational center, denoted by:

$$e(t) \in \mathcal{E}.$$

For each center $e \in \mathcal{E}$, define the subset of its professors:

$$T_e = \{t \in T \mid e(t) = e\}.$$

The mapping $e : T \to \mathcal{E}$ is a fixed input parameter of the model and is not part of the decision process.

For each professor $t \in T$, let C_t be the supervision capacity of professor t, defined as the maximum number of internships that they can supervise according to their available teaching hours. It is assumed that $C_t > 0$ for every professor $t \in T$, and that each educational center has at least one professor, i.e., $|T_e| > 0$ for all $e \in \mathcal{E}$.

In addition, the instance is assumed to be structurally feasible. In particular, the number of available placements and the total supervision capacity must be sufficient to assign all students, and company quotas and placement interdependencies must be mutually compatible. Therefore, the following basic conditions are assumed to hold:

$$|P| \geq |S|, \qquad \sum_{k \in B} q_k \leq |S| \leq \sum_{k \in B} Q_k, \qquad \sum_{t \in T} C_t \geq |S|.$$

These conditions are necessary feasibility conditions, although the actual feasibility of an instance also depends on the compatibility between company quotas and interdependent placement groups.

To model supervision-related spatial dispersion and travel effort, we assume a precomputed and normalized distance matrix. Let d_{uv} denote the normalized distance between two locations u and v, used to model supervision effort. Distances are assumed to be given by a precomputed distance matrix, where each location belongs to the set of relevant points composed of professors' educational centers and internship locations. In particular,

$$u, v \in \{o_t \mid t \in T\} \cup P,$$

where o_t denotes the geographic location of the educational center $e(t)$ to which professor t belongs. For notation simplicity, each placement $j \in P$ is identified with its geographic location when computing supervision-related distances.

Distances are normalized using a global min–max scaling over the complete set of relevant point pairs of the instance:

$$d_{uv} = \frac{\tilde{d}_{uv} - d_{\min}}{d_{\max} - d_{\min}},$$

where $\tilde{d}_{uv}$ denotes the original non-normalized distance, and $d_{\min}$ and $d_{\max}$ denote respectively the minimum and maximum distances computed over all relevant pairs of locations. If $d_{\max} = d_{\min}$, all normalized supervision distances are set to zero.

This normalization is performed once per instance as a preprocessing step. As a result, $d_{uv} \in [0, 1]$ for all relevant pairs, ensuring that supervision-related distance terms are comparable within the instance while preserving the relative spatial structure of the problem.

The distance matrix underlying the values d_{uv} is constructed as a preprocessing step prior to solving the allocation problem, based on the available geographic information. Distances may be computed using Euclidean metrics, road network distances, or estimated travel times, depending on the desired level of modeling detail.

In the original SIPA model, the distance between a student and a placement is normalized independently for each student using a min–max scaling over the set of placements available to that student. This student-level normalization is preserved when computing the utilities τ_{ij}, as it reflects individual commuting perception.

In contrast, supervision-related distances are normalized globally at the instance level using all relevant center–placement and placement–placement pairs. This global normalization is adopted because supervision effort represents a structural and aggregated organizational cost rather than an individual preference. Consequently, individual normalization for student utilities and global normalization for supervision dispersion coexist in the model as two distinct normalization schemes.

Extended decision variable. To link students, placements, and supervisors, the original assignment variable is extended as follows:

$$x_{ijt} = \begin{cases} 1, & \text{if student } i \text{ is assigned to placement } j \text{ under} \\ & \text{the supervision of professor } t, \\ 0, & \text{otherwise,} \end{cases} \qquad \forall i \in S,\ j \in P,\ t \in T. \tag{1}$$

Aggregated supervision effort and workload balance. In order to evaluate supervision effort at the professor level, we define several derived quantities that capture the geographical dispersion of supervised placements and the workload distribution among professors.

For each professor $t \in T$, let A_t denote the set of locations that must be covered:

$$A_t = \{o_t\} \cup \left\{ j \in P : \sum_{i \in S} x_{ijt} = 1 \right\}, \qquad \forall t \in T. \tag{2}$$

Let L_t denote the length of the minimum spanning tree over the nodes in A_t, which provides a compact proxy for spatial dispersion without explicitly defining supervision routes:

$$L_t = \sum_{(u,v)\in \mathrm{MST}(A_t)} d_{uv}, \qquad \forall t \in T. \tag{3}$$

Let N_t denote the supervision load of professor t, defined as the number of supervised students or internships:

$$N_t = \sum_{i\in S} \sum_{j\in P} x_{ijt}, \qquad \forall t \in T. \tag{4}$$

For each professor $t \in T$, the normalized workload proportion is defined as:

$$p_t = \frac{N_t}{C_t}, \qquad \forall t \in T. \tag{5}$$

For each educational center $e \in \mathcal{E}$, let $\bar{p}_e$ denote the average workload proportion of its professors:

$$\bar{p}_e = \frac{1}{|T_e|} \sum_{t\in T_e} p_t, \qquad \forall e \in \mathcal{E}. \tag{6}$$

The workload imbalance within each educational center is measured through the standard deviation of workload proportions:

$$\sigma_e = \sqrt{\frac{1}{|T_e|} \sum_{t\in T_e} (p_t - \bar{p}_e)^2}, \qquad \forall e \in \mathcal{E}. \tag{7}$$

Finally, the average supervision dispersion cost per professor is defined as:

$$\bar{L}_t = \begin{cases} \dfrac{L_t}{N_t}, & \text{if } N_t > 0, \\ 0, & \text{if } N_t = 0, \end{cases} \qquad \forall t \in T. \tag{8}$$

Since $d_{uv} \in [0, 1]$ and the minimum spanning tree over $|A_t| = 1 + N_t$ nodes contains at most N_t edges, each bounded above by 1, it follows that $L_t \leq N_t$. Therefore, for $N_t > 0$, $\bar{L}_t \leq 1$, and consequently $\bar{L}_t \in [0, 1]$. When $N_t = 0$, we define $\bar{L}_t = 0$, since no supervision effort is incurred.

Under the current definitions, the inclusion of $\bar{L}_t$ and σ_e introduces non-linear dependencies, yielding a mixed-integer non-linear formulation.

3.3 Constraints

The introduction of academic supervisors requires redefining the original SIPA constraints in order to account for the extended decision variable x_{ijt}. While the conceptual role of these constraints remains unchanged, their formulation is adapted to ensure consistency with the student–placement–professor assignment structure. In what follows, constraints (R1)–(R6) correspond to adapted versions of the original SIPA model, whereas constraint (R7) arises specifically from the incorporation of professors.

For the interdependency constraints, an auxiliary binary variable $\delta_I \in \{0, 1\}$ is used for each group $I \in \mathcal{I}$, indicating whether the corresponding group of placements is activated.

(R1) Complete assignment of students. Each student must be assigned to exactly one internship placement under the supervision of exactly one professor:

$$\sum_{j\in P} \sum_{t\in T} x_{ijt} = 1, \qquad \forall i \in S. \tag{9}$$

(R2) Placement capacity. Each internship placement can be assigned to at most one student:

$$\sum_{i \in S} \sum_{t \in T} x_{ijt} \leq 1, \qquad \forall j \in P. \tag{10}$$

(R3) Maximum company quota. At most Q_k students can be assigned to placements offered by company $k \in B$:

$$\sum_{i \in S} \sum_{j \in P_k} \sum_{t \in T} x_{ijt} \leq Q_k, \qquad \forall k \in B. \tag{11}$$

(R4) Minimum company quota. At least q_k students must be assigned to placements offered by company $k \in B$:

$$\sum_{i \in S} \sum_{j \in P_k} \sum_{t \in T} x_{ijt} \geq q_k, \qquad \forall k \in B. \tag{12}$$

(R5) Interdependent placements – upper bound. Placements belonging to the same interdependency group must be assigned jointly:

$$\sum_{i \in S} \sum_{j \in I} \sum_{t \in T} x_{ijt} \leq |I|\, \delta_I, \qquad \forall I \in \mathcal{I}. \tag{13}$$

(R6) Interdependent placements – lower bound. If an interdependency group is activated, all its placements must be assigned:

$$\sum_{i \in S} \sum_{j \in I} \sum_{t \in T} x_{ijt} \geq |I|\, \delta_I, \qquad \forall I \in \mathcal{I}. \tag{14}$$

(R7) Maximum supervision capacity. Each professor can supervise at most C_t students or internships:

$$\sum_{i \in S} \sum_{j \in P} x_{ijt} \leq C_t, \qquad \forall t \in T. \tag{15}$$

3.4 Multi-objective formulation

The incorporation of academic supervisors and educational centers introduces additional organizational criteria that must be considered in the optimization process. While the original SIPA objective function captures the preferences and objectives associated with students and companies, the extended model also accounts for supervision-related organizational criteria, such as the normalized geographical dispersion of internship assignments at the professor level and the proportional workload balance within each educational center.

Accordingly, the modified problem is formulated as a multi-objective optimization problem with four objective components:

- maximize the overall utility of students:

$$f_1 = \sum_{i \in S} \sum_{j \in P} \sum_{t \in T} \tau_{ij}\, x_{ijt};$$

- maximize the overall company-side utility:

$$f_2 = \sum_{i \in S} \sum_{j \in P} \sum_{t \in T} \eta_{ij}\, x_{ijt};$$

- minimize the total supervision dispersion cost:

$$f_3 = \sum_{t \in T} \bar{L}_t;$$

- minimize the total workload imbalance across educational centers:

$$f_4 = \sum_{e \in \mathcal{E}} \sigma_e.$$

Let $\mathcal{X}$ denote the feasible set defined by constraints (R1)–(R7). Thus, the optimization problem can be expressed as:

$$\max_{x \in \mathcal{X}} \left(f_1(x), f_2(x), -f_3(x), -f_4(x) \right), \tag{16}$$

where $\tau_{ij} \in [0,1]$ denotes the utility of assigning student i to placement j, and $\eta_{ij} \in [0,1]$ denotes the utility of placement j or its associated company when assigned student i.

Specifically, the objective component f_3 represents the total normalized professor-level dispersion score. For each professor, the supervision dispersion is estimated through the average minimum-spanning-tree length associated with the internships under their supervision. Lower values indicate more geographically compact supervision configurations and therefore lower supervision effort.

Similarly, the objective component f_4 represents the total workload imbalance across educational centers. For each center, workload imbalance is measured as the standard deviation of the normalized workload proportions of its professors. Lower values indicate a more balanced distribution of supervision duties among professors belonging to the same educational center.

When a scalar formulation is required, the four components can be combined through a weighted sum:

$$\max_{x \in \mathcal{X}} \ \lambda_1 \sum_{i \in S} \sum_{j \in P} \sum_{t \in T} \tau_{ij}\, x_{ijt} \ + \ \lambda_2 \sum_{i \in S} \sum_{j \in P} \sum_{t \in T} \eta_{ij}\, x_{ijt} \ - \ \lambda_3 \sum_{t \in T} \bar{L}_t \ - \ \lambda_4 \sum_{e \in E} \sigma_e. \tag{17}$$

The weighting parameters λ_1, λ_2, λ_3 and λ_4 control the relative importance of the four objective components and are required to satisfy:

$$\lambda_1 + \lambda_2 + \lambda_3 + \lambda_4 = 1, \qquad \lambda_1, \lambda_2, \lambda_3, \lambda_4 \in [0,1]. \tag{18}$$

Since the four objective components are aggregated over different sets, their numerical scales may differ across instances. Therefore, whenever the weighted formulation is used for comparison or auxiliary scalarization, the objective components should be normalized consistently.

In this work, the problem is addressed from a Pareto-based multi-objective perspective. Therefore, the weighted formulation is not used as the main optimization criterion, but it may be used as an auxiliary scalarization mechanism when required by specific procedures or analyses.

4 Proposed Method: HH-LAP-SIPA

The problem formulated in the previous section combines student–placement assignment decisions with academic supervision decisions, company quotas, professor supervision capacities,

placement interdependencies, and several potentially conflicting objectives. This structure increases the complexity of the search space and hinders the application of exact approaches to larger instances.

For this reason, this section presents HH-LAP-SIPA, a multi-objective metaheuristic approach designed to address this modified problem. The method is described as a general algorithmic proposal and, subsequently, two variants are introduced. Both variants share the same basic structure but differ in the way the placement–professor assignment layer is constructed.

4.1 General overview

HH-LAP-SIPA is conceived as a hybrid heuristic for the Student–Industry Placement Allocation problem with academic supervision. Its name reflects the combination of an iterative process for generating feasible solutions with a procedure based on linear assignment problems, applied specifically to the student–placement assignment layer. The objective of the method is not to optimize a single aggregated function, but to generate a diverse set of feasible and non-dominated solutions representing different trade-offs among the four objectives of the model.

The general idea of the method is to decompose the construction of complete solutions into three layers. First, a feasible subset of placements P' is randomly selected, with a cardinality equal to the number of students, that is, $|P'| = |S|$. This selection must satisfy the structural constraints associated with placements and companies, including minimum and maximum company quotas and placement interdependencies.

The aim of this phase is not to directly select the best placements in isolation, but to generate structurally feasible subsets that can later be evaluated when combined with student and professor assignments.

Once the subset P' has been fixed, the method addresses two complementary decisions:

- The placement–professor assignment, which assigns an academic supervisor to each selected placement while respecting the maximum supervision capacities of professors. This layer is responsible for the organizational components of the model, mainly the geographical dispersion of supervision and the workload balance among professors.

  The main difference between the two algorithmic variants lies in this layer.

- The student–placement assignment, which is solved through one-to-one linear assignment problems, since the number of selected placements coincides with the number of students.

Thus, for each feasible subset of placements, HH-LAP-SIPA solves two independent LAPs. The first one maximizes student utility, generating a student–placement assignment oriented toward f_1. The second one maximizes company-side utility, generating a student–placement assignment oriented toward f_2. Therefore, the method does not impose a single intermediate student–placement assignment, but explicitly explores the two main perspectives of the student–placement allocation layer.

Complete solutions are obtained by combining the subset of placements P', a placement–professor assignment, and one of the two student–placement assignments obtained through LAP. The construction process is supported by feasibility-preserving procedures and repair mechanisms, which are used to handle structural constraints such as company quotas, placement interdependencies, and maximum supervision capacities. Each generated solution is validated against the constraints of the modified model and evaluated according to the four components of the multi-objective formulation.

Next, feasible solutions are compared using Pareto dominance [9], and an external archive is maintained with the non-dominated solutions found during the search. Therefore, HH-LAP-SIPA does not return a single final assignment, but rather a set of alternatives that can later be analyzed by the educational institution according to its priorities.

Two variants of the method are defined over this general structure:

- **HH-LAP-SIPA-Lite**: this variant uses a lightweight construction of the placement–professor layer. For each feasible subset P', a single placement–professor assignment is generated and combined with the two student–placement LAPs. Therefore, each iteration can produce up to two complete solutions: one student-oriented solution and one company-oriented solution. This variant reduces the computational cost per iteration and favors the exploration of a larger number of placement subsets under a fixed time budget.

- **HH-LAP-SIPA-Intensive**: this variant intensifies the construction of the placement–professor layer. Instead of generating a single supervisor assignment for each subset P', it constructs several candidate assignments using target professor loads, spatial grouping of placements, and path relinking between assignments. Each placement–professor assignment is combined with the two student–placement LAPs, so the same iteration can produce more than two complete solutions. This variant aims to explore the academic supervision dimension in greater depth, especially in terms of geographical dispersion and workload balance, although at a higher computational cost per iteration.

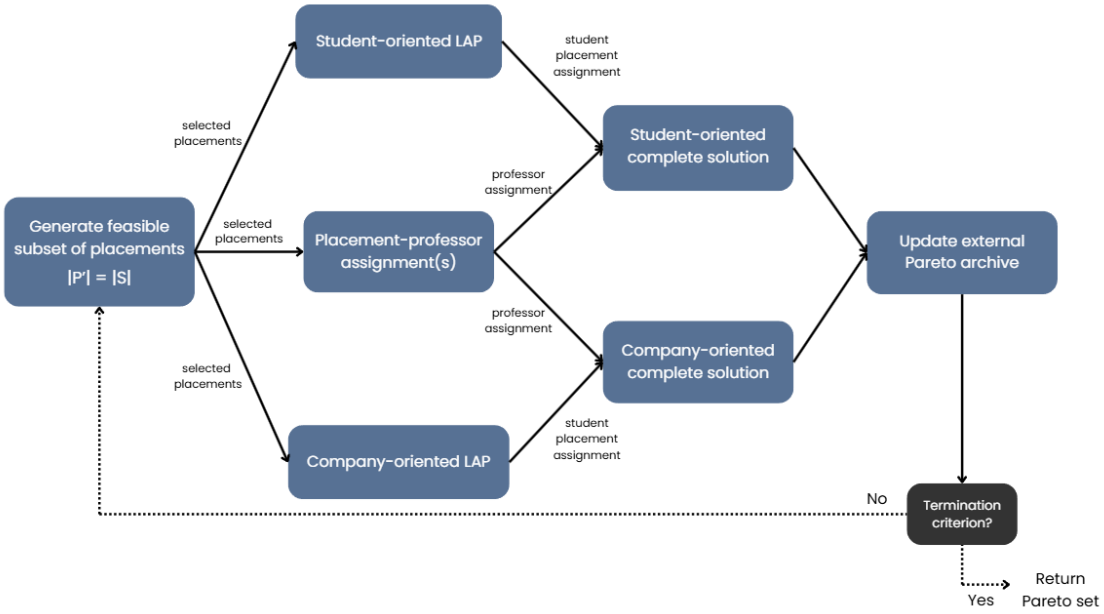


Figure 1: General workflow of the HH-LAP-SIPA framework.

4.2 Solution representation

Each solution generated by HH-LAP-SIPA represents a complete assignment for the problem, including both the relationship between students and placements and the relationship between placements and professors. Although the mathematical formulation can be expressed through binary variables x_{ijt}, which indicate whether student i is assigned to placement j

under the supervision of professor t, the implementation uses a more compact representation based on two assignment layers. This representation avoids explicitly storing a three-dimensional student–placement–professor structure and facilitates the construction, repair, validation, and evaluation processes of the solutions generated by the heuristic.

The first layer corresponds to the student–placement assignment. It is represented by a vector a, where $a_i = j$ indicates that student i is assigned to placement j. Thus, each position of the vector corresponds to a student, and its value identifies the placement occupied by that student. This representation makes it possible to directly verify that all students have been assigned and that each student receives exactly one placement.

The second layer corresponds to the placement–professor assignment. It is represented by a vector b, where $b_j = t$ indicates that placement j is supervised by professor t. This assignment is only relevant for placements that have been selected and occupied in the solution. Therefore, a complete assignment can be interpreted as the combination of both layers: if $a_i = j$ and $b_j = t$, then student i is assigned to placement j under the supervision of professor t.

In addition to these two main vectors, the solution maintains several auxiliary structures that make it possible to efficiently check feasibility during solution construction, repair, and evaluation. First, the load of each company is stored, indicating how many placements associated with that company have been occupied and allowing the minimum and maximum agreed quotas to be verified. Second, the load of each professor is stored, indicating how many placements they supervise in the current solution and allowing their maximum supervision capacity to be checked. Finally, an occupancy indicator is maintained for each placement, specifying whether the placement has been selected and assigned to a student.

4.3 Feasible solution generation and repair

A central aspect of HH-LAP-SIPA is that candidate solutions must be feasible before being evaluated and incorporated into the external Pareto archive. For this reason, the method does not rely on generating arbitrary solutions and subsequently penalizing violated constraints. Instead, solutions are constructed progressively, preserving or recovering feasibility at each decision layer.

This strategy enables the method to explore different feasible regions of the search space while avoiding the evaluation of structurally invalid assignments. Moreover, the feasible solutions generated through this process provide valid starting points on which the subsequent algorithmic components of HH-LAP-SIPA can operate.

The first step consists of generating a subset of placements P' such that $|P'| = |S|$. This condition ensures that, once this subset has been fixed, a one-to-one assignment between students and placements can be constructed. The selection of P' must also satisfy the structural constraints associated with companies, so that the number of selected placements from each company lies between its minimum quota q_k and its maximum quota Q_k.

Placement interdependencies are also handled during this phase. If a group of placements is subject to an interdependency relation, the whole group must either be selected or excluded. Therefore, solutions in which a block of interdependent placements is only partially activated are not allowed. To this end, the initial construction may randomly select dependency blocks and subsequently apply a repair procedure that activates or deactivates blocks until a selection compatible with the total number of students, company quotas, and the availability of non-dependent placements is obtained. If the selection cannot be repaired, the construction attempt is discarded and a new subset is generated.

Once a structurally feasible subset P' has been obtained, the student–placement assign-

ment layer is constructed. Since the number of selected placements is equal to the number of students, this layer can be solved as a one-to-one linear assignment problem. In this way, each student receives exactly one placement and each selected placement is occupied by exactly one student, avoiding feasibility issues in this decision layer.

Another layer corresponds to the placement–professor assignment, where each occupied placement must receive exactly one academic supervisor. This assignment is constructed while respecting the maximum supervision capacity of each professor, so that new placements can only be assigned to professors with remaining capacity. If, during construction, a placement cannot be directly assigned to a professor, repair mechanisms based on admissible reassignments are applied, provided that they keep all placements supervised and do not cause any professor to exceed their maximum capacity. If a feasible professor assignment still cannot be obtained, the candidate solution is discarded.

Repair mechanisms are therefore used to recover or preserve feasibility when a partial construction does not satisfy some constraint. In the placement selection layer, repair operates on dependency blocks and on the number of placements selected from each company. In the professor assignment layer, repair operates on the distribution of placements among supervisors, while maintaining the mandatory condition that every occupied placement has an assigned professor and that maximum supervision capacities are respected.

Finally, before evaluating a complete solution, a global validation step is applied. This validation checks that all students are assigned, that minimum and maximum company quotas are satisfied, that no interdependent placement block is partially selected, that every occupied placement has an assigned professor, and that no professor exceeds their maximum supervision capacity. Only solutions that pass this validation step are evaluated according to the four objectives of the model and considered for insertion into the external Pareto archive.

This feasibility logic is common to both variants of the heuristic. The difference between them does not lie in the constraints that solutions must satisfy, but rather in the specific way in which the placement–professor layer is constructed and explored. These differences are described in the subsections corresponding to each variant.

4.4 Multi-objective evaluation and Pareto archive

Once a complete solution has been constructed and validated, HH-LAP-SIPA evaluates it according to the four objectives defined in the model. The evaluation is performed in a multi-objective manner, keeping each component separate rather than necessarily combining them into a single scalar function. Thus, each solution is represented by an objective vector composed of the total student utility, the total company-side utility, the geographical dispersion of supervision, and the workload imbalance among professors.

The first two components, associated with students and companies, are to be maximized. In particular, τ_{sum} measures the total utility obtained from the students' perspective, whereas η_{sum} measures the total utility obtained from the perspective of companies or placements. In contrast, the last two components are to be minimized. The value $\bar{L}_{\text{sum}}$ represents the sum of the average geographical supervision dispersion costs, whereas σ_{sum} measures the aggregate workload imbalance among professors within educational centers.

Therefore, each solution s is evaluated through the following objective vector:

$$F(s) = \left(\tau_{\text{sum}}(s), \eta_{\text{sum}}(s), \bar{L}_{\text{sum}}(s), \sigma_{\text{sum}}(s)\right),$$

where the first two components are maximized and the last two are minimized. This representation allows solutions to be compared without imposing a fixed preference among the different objectives of the problem.

Although an auxiliary scalar value can also be computed through a weighted combination of the four components, corresponding to the scalar formulation introduced as the objective function in the problem definition,

$$z(s) = \lambda_1 \tau_{\text{sum}}(s) + \lambda_2 \eta_{\text{sum}}(s) - \lambda_3 \bar{L}_{\text{sum}}(s) - \lambda_4 \sigma_{\text{sum}}(s),$$

this value is not used to determine Pareto dominance. Its role is only complementary, since it provides an aggregated measure that can be useful for result analysis, while the main comparison criterion remains multi-objective dominance.

Solutions are compared using Pareto dominance. Given two valid solutions s_a and s_b, s_a is said to dominate s_b if s_a is at least as good as s_b in all objectives and strictly better in at least one of them. That is,

$$\begin{aligned} \tau_{\text{sum}}(s_a) \geq \tau_{\text{sum}}(s_b), \quad \eta_{\text{sum}}(s_a) \geq \eta_{\text{sum}}(s_b), \\ \bar{L}_{\text{sum}}(s_a) \leq \bar{L}_{\text{sum}}(s_b), \quad \sigma_{\text{sum}}(s_a) \leq \sigma_{\text{sum}}(s_b), \end{aligned}$$

with at least one strict inequality.

During the search, HH-LAP-SIPA maintains an external archive of non-dominated solutions. Whenever a new valid solution is generated, it is compared with the solutions already stored in the archive: if the new solution is dominated by an existing solution, or if it is equivalent to a solution already stored, it is discarded; if it dominates one or more solutions in the archive, those solutions are removed; and if it is neither dominated nor equivalent to any existing solution, it is inserted into the external archive. In this way, the external archive retains only the non-dominated solutions found during the search.

The final output of the algorithm is not a single assignment, but a set of non-dominated solutions representing different trade-offs among the objectives of the problem. This structure allows the educational institution to subsequently analyze the available alternatives and select the most appropriate solution according to its decision criteria.

4.5 LAP-based local assignment procedure

The local assignment procedure of HH-LAP-SIPA is applied exclusively to the student–placement layer. Once a feasible subset of placements P' and a feasible placement–professor assignment have been fixed, the remaining decision consists of determining which student occupies each selected placement. Since the number of students coincides with the number of selected placements, this decision can be formulated as a one-to-one linear assignment problem [10].

This procedure is considered local because it optimizes the student–placement assignment within a previously fixed subset of placements, without modifying either the selection P' or the assignment of professors. Therefore, it does not operate on the entire solution structure, but only on the layer in which utilities directly depend on the student–placement pair.

For each fixed subset P', HH-LAP-SIPA builds two student–placement benefit matrices. Each row represents a student and each column represents one of the selected placements in P'. The first matrix contains the values τ_{ij} and is used to obtain a student-oriented assignment, while the second contains the values η_{ij} and is used to obtain a company-oriented assignment.

From these matrices, two independent LAPs are solved. The first one maximizes the sum of the τ_{ij} utilities, whereas the second one maximizes the sum of the η_{ij} utilities. Conceptually, these problems can be expressed as:

$$\max \sum_{i \in S} \sum_{j \in P'} \tau_{ij} y_{ij} \qquad \text{and} \qquad \max \sum_{i \in S} \sum_{j \in P'} \eta_{ij} y_{ij}$$

subject to the one-to-one assignment constraints:

$$\begin{aligned} \textstyle\sum_{j \in P'} y_{ij} &= 1, && \forall i \in S; \\ \textstyle\sum_{i \in S} y_{ij} &= 1, && \forall j \in P'; \\ y_{ij} &\in \{0,1\}, && \forall i \in S,\ \forall j \in P'. \end{aligned}$$

The first constraint ensures that each student is assigned to exactly one placement, while the second ensures that each selected placement is occupied by exactly one student. Thus, each LAP produces a complete and feasible student–placement assignment within the subset P'.

In the implementation, these assignment problems are solved using the `lapjv` solver, based on the Jonker–Volgenant algorithm [11]. Since the solver is formulated as a minimization problem, the maximization of the benefit matrices is transformed into minimization by changing the sign of the corresponding matrix. Thus, maximizing the τ_{ij}-based and η_{ij}-based benefit matrices is equivalent to minimizing their negated versions, respectively.

The output of the local procedure consists of two student–placement assignments for the same subset P'. Each of these assignments is then combined with the previously constructed placement–professor assignment to generate complete solutions. In the Lite variant, the two LAPs are combined with a single placement–professor assignment, whereas in the Intensive variant, the two LAPs are solved only once for each subset P' and reused for all placement–professor assignments generated during the intensification process.

This use of LAP introduces an exact optimization component within the heuristic. Although HH-LAP-SIPA does not solve the complete problem exactly, since placement selection and professor assignment are handled through heuristic procedures, it does obtain an optimal student–placement assignment for each feasible subset considered and for each of the two benefit matrices used.

4.6 Algorithm variants

Two variants of HH-LAP-SIPA are considered in order to analyze different ways of constructing the placement–professor assignment layer. Both variants share the same general architecture described in the previous subsections: they generate a feasible subset of placements P', construct one or more feasible placement–professor assignments, solve two student–placement LAPs oriented toward student and company-side utility, evaluate the resulting complete solutions using the four objectives of the model, and update the external Pareto archive with the non-dominated solutions found during the search.

The main difference between the two variants lies in the degree of intensification applied to the placement–professor assignment layer. HH-LAP-SIPA-Lite follows a lightweight strategy that constructs a single feasible placement–professor assignment for each subset P', allowing the method to perform more iterations and explore more placement subsets under a fixed time budget. In contrast, HH-LAP-SIPA-Intensive devotes more computational effort to the same subset P' by generating several placement–professor assignments, with the aim of exploring the supervision dimension in greater depth. Therefore, comparing both variants makes it possible to study the trade-off between broader exploration of feasible placement subsets and deeper intensification of supervision configurations.

4.6.1 HH-LAP-SIPA-Lite

HH-LAP-SIPA-Lite is the lightweight variant of the proposed method. Its objective is to keep the construction of complete solutions computationally inexpensive, so that a larger number of feasible placement subsets can be explored under a fixed time budget. For this reason, the variant performs a single placement–professor assignment for each feasible subset P', and then reuses this assignment when constructing the two complete solutions obtained from the student–placement LAPs.

Once a feasible subset P' has been generated, the Lite variant constructs the placement–professor assignment through a guided placement-by-placement procedure. The selected placements are processed one by one, and for each placement the algorithm considers only professors with remaining supervision capacity. Among the feasible candidates, the assignment is guided by the estimated incremental effect on the supervision-related components of the model, namely the geographical dispersion of supervision and the workload balance among professors.

Feasibility is maintained during this construction by ensuring that no professor exceeds their maximum supervision capacity and that every selected placement receives exactly one academic supervisor. If, at any point, no professor with available capacity can be assigned to a selected placement, or if the final placement–professor assignment does not satisfy the feasibility conditions, the current subset P' is discarded and the algorithm proceeds to a new iteration.

After a feasible placement–professor assignment has been obtained, the Lite variant solves the two student–placement LAPs described in the previous subsection. Since the placement–professor assignment is fixed and reused, each feasible subset P' can produce up to two complete solutions (one combining the supervisor assignment with the student-oriented LAP and another combining it with the company-oriented LAP).

Each complete solution is then validated, evaluated according to the four objectives of the model, and compared with the solutions stored in the external Pareto archive. Therefore, the Lite variant favors broader exploration of the search space, since its lower computational cost per iteration can allow it to evaluate a larger number of feasible placement subsets, although it explores only one supervision configuration for each subset.

4.6.2 HH-LAP-SIPA-Intensive

HH-LAP-SIPA-Intensive is the intensified variant of the proposed method. Unlike the Lite variant, which constructs a single placement–professor assignment for each feasible subset P', this variant generates several placement–professor assignments for the same subset. The objective is to explore the supervision layer in greater depth, especially with respect to geographical dispersion and workload balance, while keeping the student–placement assignment layer fixed through the two LAPs associated with student and company-side utilities.

For each feasible subset P', the Intensive variant first computes target professor loads. These target loads provide a reference distribution of the selected placements among professors and are defined proportionally to their maximum supervision capacities. In this way, professors with higher supervision capacity can receive a higher target load, while the construction still aims to avoid excessive imbalance in the final assignment.

The selected placements are then organized into spatial groups. In the implementation, these groups are built from the companies associated with the placements, so that placements belonging to the same company are treated as part of the same active cluster when they appear in P'. This grouping is used to favor supervision configurations in which geographically related placements are assigned coherently, reducing the dispersion associated with academic

supervision.

Using the active clusters of P', the algorithm constructs an initial placement–professor assignment. This initial assignment is guided by spatial compactness and feasibility, which means that clusters are processed and assigned to professors with available capacity, taking into account the proximity between professors and the corresponding placements, as well as the remaining target load of each professor. If a complete feasible initial assignment cannot be obtained, the current subset P' is discarded and the algorithm proceeds to a new iteration.

In addition to the initial assignment, the Intensive variant constructs a guide assignment. This guide assignment is also required to be feasible, but it is oriented more explicitly toward the target professor loads. Its purpose is to provide an alternative supervision configuration that represents a more balanced distribution of placements among professors, while still respecting maximum supervision capacities.

Once both assignments have been obtained, path relinking is applied between the initial assignment and the guide assignment [12]. Starting from the initial assignment, the algorithm progressively modifies the professor assigned to selected placements in order to move toward the guide assignment. Each accepted intermediate assignment defines a new placement–professor configuration for the same subset P'. During this process, only feasible moves are accepted (each placement must remain supervised, and no professor is allowed to exceed their maximum supervision capacity). If no admissible move is available, the path relinking process stops.

The resulting placement–professor assignments are then combined with the two student–placement LAPs. Since these LAPs depend only on the selected placements and on the student–placement utility matrices, they are solved once for each subset P' and reused for all placement–professor assignments generated during the intensification process. Consequently, each supervision configuration can be combined with both the student-oriented and the company-oriented LAP assignment, producing multiple complete candidate solutions from the same subset P'.

Each candidate solution is validated, evaluated according to the four objectives of the model, and compared with the solutions stored in the external Pareto archive. Therefore, HH-LAP-SIPA-Intensive favors a deeper exploration of the supervision dimension, with the aim of obtaining more refined configurations in terms of geographical dispersion and workload balance. This comes at the cost of a higher computational effort per iteration, which may reduce the number of different placement subsets explored under the same time budget.

4.7 Algorithm overview

After defining the main components of HH-LAP-SIPA and its two variants, Algorithm 1 summarizes the common execution flow of the method. The only variant-dependent step is the generation of placement–professor assignments, which is represented through a generic function in order to avoid repeating two separate algorithms.

Algorithm 1 General HH-LAP-SIPA framework

Require: Problem instance, variant $v \in \{\text{Lite}, \text{Intensive}\}$, stopping criterion
Ensure: External Pareto archive $\mathcal{A}$

$\mathcal{A} \leftarrow \emptyset$
while the stopping criterion is not met **do**
  Randomly generate a feasible subset P' such that $|P'| = |S|$
  $Y^{\tau} \leftarrow \text{StudentLAP}(P')$
  $Y^{\eta} \leftarrow \text{CompanyLAP}(P')$
  $\mathcal{R} \leftarrow \text{ProfAssign}(P', v)$
  for each $r \in \mathcal{R}$ **do**
    for each $Y \in \{Y^{\tau}, Y^{\eta}\}$ **do**
      Construct a complete solution s from P', r, and Y
      if s is feasible **then**
        Evaluate s according to the four objectives
        Update $\mathcal{A}$ with s
      end if
    end for
  end for
end while
return $\mathcal{A}$

5 Experimental Setup

This section describes the experimental setup used to evaluate the proposed HH-LAP-SIPA approach. The objective of the experiments is to assess the ability of the proposed method to generate high-quality Pareto fronts for the SIPA problem addressed in this work.

To this end, HH-LAP-SIPA is compared against several reference algorithms over a set of synthetic instances of different sizes. The experimental design considers multiple independent executions per instance and algorithm, fixed computational budgets, and a common multi-objective evaluation methodology based primarily on the hypervolume indicator.

5.1 Problem instances

The experimental evaluation was carried out using a set of synthetic instances generated for the SIPA problem addressed in this work. These instances were designed to represent allocation scenarios of increasing size while preserving the main structural elements of the problem: students, companies, internship placements, academic supervisors, educational centers, company quotas, professor supervision capacities, placement dependencies, competence information, and distance information.

A total of 20 instances were generated, grouped into five experimental families according to the number of students (200, 400, 600, 800, and 1000 students), with four independent instances generated for each size. In all cases, a placement surplus of 20% was considered, meaning that the number of available placements was set to 20% above the number of students. This surplus provides additional allocation flexibility while maintaining a constrained assignment scenario. Therefore, the number of placements ranged from 240 in the smallest instances to 1200 in the largest ones.

Each instance includes a set of companies offering internship placements, where each company is associated with minimum and maximum allocation quotas. Placement dependencies were also generated, representing groups of placements that must be either selected

together or not selected at all. In addition, the instances include educational centers and professors, with each professor assigned to one center and associated with a maximum supervision capacity. Moreover, a slack ratio of 0.2 was used when generating company quotas and professor supervision capacities, introducing an additional capacity margin while preserving the constrained nature of the allocation problem.

The competence space was organized into eight academic profile blocks, each composed of six base competences. For each placement, a competence-relevance vector was generated, with two competences identified as the main requirements of the placement. For each student, the instance includes a competence development vector, representing the student's current competence level, and a competence preference vector, representing the competences that the student would prefer to develop during the internship. Student heterogeneity was introduced through different academic profiles, including excellent and notable profiles generated with probabilities of 0.05 and 0.25, respectively.

Distance information was generated synthetically for the three matrices required by the model (student–company, center–company, and company–company distances). These matrices support both student-related utility calculations and supervision-related organizational criteria, including the estimation of geographical dispersion among the placements supervised by each professor. The synthetic distance values were generated from a normal distribution with mean 1500 and standard deviation 250.

Table 1 summarizes the main characteristics of the generated instances. It reports, for each family of instances, the number of students ($|S|$), placements ($|P|$), and the ranges for companies ($|B|$), educational centers ($|E|$), professors ($|T|$), and dependency groups ($|I|$). Ranges are computed over the four independent instances generated for each size.

Table 1: Summary of the synthetic problem instances.

$\|S\|$	$\|P\|$	$\|B\|$	$\|E\|$	$\|T\|$	$\|I\|$
200	240	26–33	2–6	2–14	26–33
400	480	53–69	2–5	3–13	26–57
600	720	77–90	2–5	2–8	77–90
800	960	117–131	5–6	8–18	117–131
1000	1200	129–170	2–6	4–7	129–170

The generation process was controlled through fixed random seeds in order to ensure reproducibility. For the instances with 200, 400, and 600 students, the base seed was set to 1234, whereas for the instances with 800 and 1000 students the base seed was set to 1250. In both cases, the seed of each instance was obtained by adding the global instance index to the corresponding base seed. Both the Python random generator and the NumPy random generator were initialized for each instance.

5.2 Compared algorithms

The experimental comparison includes five algorithms designed or adapted to solve the SIPA problem addressed in this work. The selected methods cover different levels of algorithmic complexity and problem-specific knowledge, ranging from a purely random feasible baseline to the two proposed HH-LAP-SIPA variants. All algorithms generate complete feasible solutions, evaluate them according to the four objectives of the model, and maintain an external Pareto archive with the non-dominated solutions found during each execution.

The compared algorithms are the following:

1. **Random feasible baseline:** this algorithm repeatedly generates complete feasible solutions without using an explicit optimization criterion. It first selects a feasible subset of placements that satisfies company quotas, placement dependencies, and single occupancy of placements, and then assigns students to the selected placements through randomized feasible assignments. Once the student–placement layer has been constructed, professors are assigned to the occupied placements while respecting supervision capacities, using repair mechanisms when necessary to restore feasibility. Each valid solution is evaluated and considered for insertion into the external Pareto archive, so this method provides a basic reference level for assessing the benefit of more informed search strategies.

2. **NSGA-III:** this algorithm is a population-based multi-objective genetic algorithm adapted to the SIPA problem [13]. Each individual is represented by a vector that stores the placement assigned to each student, while the professor assignment is not encoded directly in the chromosome but kept as part of the associated complete solution and reconstructed after crossover and mutation in order to reduce the complexity of the representation. The initial population is composed of feasible solutions, and genetic variation is followed by repair procedures that preserve or recover feasibility in both the student–placement and placement–professor layers. Individuals are evaluated using the four objective components, with student and company utilities maximized and supervision dispersion and workload imbalance minimized. The population evolves through NSGA-III selection based on non-dominated sorting and reference points, while an external Pareto archive stores all non-dominated solutions found during the run.

3. **GRASP:** this algorithm is a constructive metaheuristic used as a stronger baseline than the purely random method [14]. It combines a greedy-randomized construction phase with a local search phase, building student–placement assignments incrementally according to a greedy score based on student utility and company-side utility, while a Restricted Candidate List introduces controlled randomness into the selection process. After a complete assignment has been constructed, the professor assignment is reconstructed and the solution is validated. The local search phase then explores swaps between pairs of students and accepts the first improving neighbour according to a normalized auxiliary scalar score that combines the four objective components. The final improved solution is evaluated and registered in the external Pareto archive, and if the greedy-randomized construction fails to produce a feasible solution within its local limits, a random feasible fallback is used to provide an initial solution before applying the same local search procedure.

4. **HH-LAP-SIPA-Lite:** this is the first variant of the proposed approach. It follows the decomposition described in Section 4, generating feasible placement subsets, constructing one supervision assignment for each subset, and solving two exact Linear Assignment Problems focused on student utility and company-side utility. This variant is included to evaluate the performance of a lightweight version of HH-LAP-SIPA, designed to exploit the efficiency of LAP solvers while keeping the supervision layer computationally simple.

5. **HH-LAP-SIPA-Intensive:** this is the second variant of the proposed approach. It keeps the same general decomposition, but performs a more intensive exploration of the supervision layer by generating multiple placement–professor assignments for each selected placement subset, using target professor loads, company-based clusters, and

path relinking. This variant is included to assess whether a richer exploration of supervision configurations improves the quality of the Pareto fronts, despite its higher computational cost.

5.3 Execution protocol

For each synthetic instance and each compared algorithm, 30 independent executions were carried out. Since the experimental set contains 20 instances and five algorithms, the complete experimental campaign consisted of 3000 executions. Each execution was initialized with a different reproducible random seed, generated from a common base seed and the corresponding algorithm, instance, and run identifier. This ensured that repetitions were independent while preserving reproducibility.

The main stopping criterion was a fixed time limit. The computational budget was defined according to the size of the instance, so that larger instances were given more execution time. The time limits were set to 300 seconds for instances with 200 students, 450 seconds for instances with 400 and 600 students, and 600 seconds for instances with 800 and 1000 students, with all algorithms receiving the same time limit when solving the same instance. No maximum number of iterations or generations was imposed as the primary stopping condition, although some algorithms used internal limits for specific phases, such as local search or constructive attempts, in order to avoid excessive time consumption inside a single iteration.

All algorithms were run under the same evaluation conditions. Student–company distances and supervision-related distances were normalized in all executions, and the external Pareto archive was not limited in size. When an algorithm required an auxiliary scalar score for internal decisions, the same weight configuration was used across the experimental campaign, with weights 0.35 for student utility, 0.35 for company-side utility, 0.20 for supervision dispersion, and 0.10 for workload imbalance. These weights were only used for internal scalar guidance or auxiliary evaluation, whereas the final comparison was based on the multi-objective Pareto fronts produced by each run.

The experiments were executed on the same virtual machine, equipped with 8 CPU cores and 16 GB of RAM. The execution process was managed by a common experimental launcher, which applied the same protocol to all algorithms and stored the resulting Pareto fronts and execution statistics in a homogeneous way.

5.4 Evaluation metrics

The main performance indicator used in the experimental evaluation was the hypervolume indicator [15], which is a standard quality measure for multi-objective optimization because it jointly captures convergence and diversity. A higher value indicates that the obtained Pareto front covers a larger dominated region of the objective space and, therefore, represents a better approximation of the trade-offs among objectives.

Hypervolume was computed independently for each problem instance. For every execution, the indicator was calculated from the external Pareto archive produced by the corresponding algorithm. Each solution in the archive was represented by the four objective components of the model (student utility τ_{sum}, company-side utility η_{sum}, supervision dispersion $\bar{L}_{\text{sum}}$, and workload imbalance σ_{sum}). Since these objectives have different scales and optimization directions, a normalization step was applied before computing the indicator.

The normalization was performed independently for each problem instance. For a given instance, the minimum and maximum values of each objective were computed using all Pareto solutions obtained by all algorithms and all independent runs on that instance. These

common bounds were then used to normalize every Pareto front associated with the same instance, ensuring that hypervolume values were comparable across algorithms and runs for that instance.

The two utility objectives, τ_{sum} and η_{sum}, were treated as maximization objectives. Therefore, their normalized values were obtained using standard min–max normalization. In contrast, supervision dispersion and workload imbalance are minimization objectives, so their normalized values were inverted after min–max scaling, making lower original values correspond to higher normalized values. After this transformation, all four normalized objectives were expressed in a common maximization-oriented space, where larger values are better.

The hypervolume was then computed over the normalized objective vectors. Since the implementation used the DEAP hypervolume routine [16], which operates under a minimization convention, the normalized maximization-oriented points were internally transformed to match this convention. After this transformation, the reference point was set using a margin of 0.1 beyond the worst value in each dimension. Finally, the resulting hypervolume was divided by the maximum possible volume induced by this reference margin, obtaining a normalized hypervolume value in the interval $[0, 1]$.

For each algorithm and instance, hypervolume was first computed independently for each of the 30 runs. Then, summary statistics were obtained by reporting the mean, standard deviation, minimum, and maximum hypervolume across runs. The mean hypervolume was used as the main comparison criterion, while the standard deviation and range provided information about the robustness and variability of each algorithm.

In addition to hypervolume, complementary indicators were considered to support the interpretation of the results. The size of the final Pareto archive was used to describe the number of non-dominated solutions generated by each run. Moreover, the individual objective values were analysed separately in order to identify whether the differences between algorithms were mainly associated with student utility, company-side utility, supervision dispersion, or workload imbalance. These complementary analyses help interpret the hypervolume results and provide a more detailed understanding of the trade-offs produced by each algorithm.

6 Results and Discussion

This section presents and discusses the experimental results obtained for the proposed SIPA problem with academic supervision. The analysis focuses on the quality of the Pareto fronts generated by the compared algorithms, using the hypervolume indicator defined in the previous section as the main performance metric. The results are then interpreted in relation to the four objective components of the model, with special attention to the behavior of the proposed HH-LAP-SIPA variants.

6.1 Overall hypervolume comparison

This subsection compares the algorithms from a global multi-objective perspective using the hypervolume indicator. For each algorithm and instance, the hypervolume was averaged over the 30 independent runs, and the global values reported in Table 2 were then computed from these 20 instance-level averages. This avoids treating repeated runs of the same instance as independent problem cases and provides a balanced global comparison across the benchmark set.

Table 2: Overall hypervolume comparison over the 20 instances.

Algorithm	Mean HV	Std. HV	Median HV	Avg. rank	Wins
HH-LAP-SIPA-Lite	0.5711	0.2942	0.7228	1.35	15
HH-LAP-SIPA-Intensive	0.3609	0.1345	0.3169	1.80	5
GRASP	0.0992	0.1808	0.0125	4.50	0
NSGA-III	0.0372	0.0272	0.0286	3.15	0
Random	0.0184	0.0057	0.0170	4.20	0

The results show a clear advantage of the two proposed HH-LAP-SIPA variants over the baseline methods. HH-LAP-SIPA-Lite obtains the highest mean hypervolume, with a value of 0.5711, followed by HH-LAP-SIPA-Intensive with 0.3609. In contrast, the best baseline method in terms of mean hypervolume, GRASP, reaches a mean hypervolume of 0.0992, while NSGA-III and the random baseline obtain considerably lower values. This gap indicates that both HH-LAP-SIPA variants are able to generate higher-quality Pareto fronts than the reference methods, with the Lite variant achieving the best overall performance.

The ranking analysis leads to the same conclusion. HH-LAP-SIPA-Lite obtains the best average rank, with a value of 1.35, while HH-LAP-SIPA-Intensive obtains the second-best average rank, with a value of 1.80. The remaining methods are clearly separated from the proposed variants, with average ranks of 3.15 for NSGA-III, 4.20 for Random, and 4.50 for GRASP. This indicates that the superiority of HH-LAP-SIPA-Lite is not only due to a few favorable instances, but is also reflected in its relative position across the full benchmark set.

The number of instance-level wins further confirms this behavior. HH-LAP-SIPA-Lite obtains the best hypervolume in 15 out of the 20 instances, while HH-LAP-SIPA-Intensive obtains the best value in the remaining 5 instances. None of the baseline methods achieves the best hypervolume in any instance. This result suggests that the structured hybrid design of the proposed methods provides a more effective search mechanism for the extended SIPA problem than the general-purpose or less structured baselines.

Figure 2 provides a visual summary of the same comparison. The proposed HH-LAP-SIPA variants clearly dominate the rest of the algorithms in terms of mean hypervolume. The error bars also show that the Lite variant presents a higher variability across instances, which is also reflected in its larger standard deviation in Table 2.

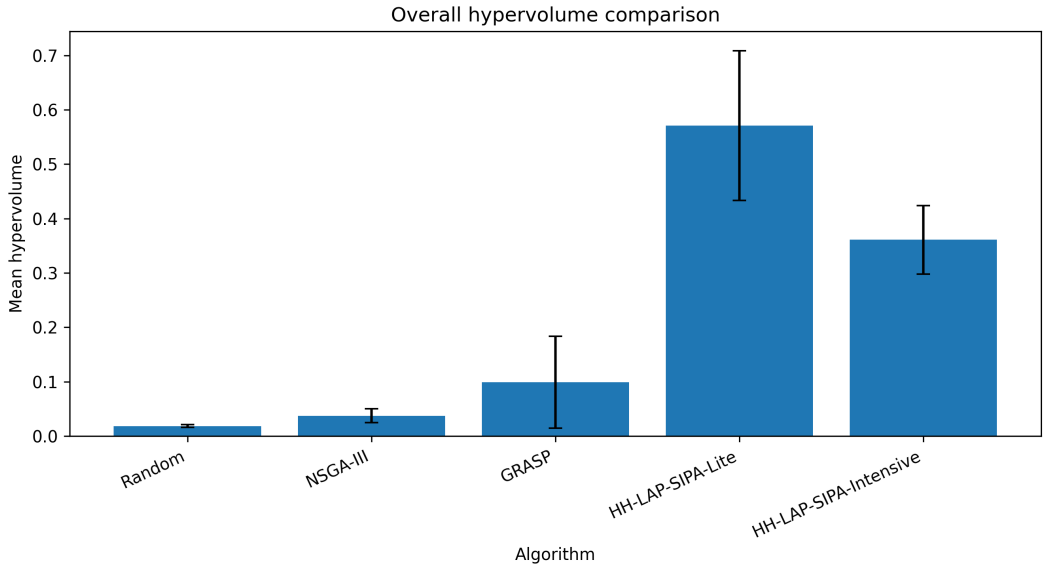


Figure 2: Overall hypervolume comparison across the complete benchmark set.

6.2 Hypervolume by instance size

This subsection analyzes the behavior of the compared algorithms as the size of the instances increases. The objective is to assess whether the global performance differences observed in the previous subsection remain stable when the number of students grows from 200 to 1000. Since each size group contains four independently generated instances, the results should be interpreted as evidence of relative behavior across increasing problem sizes, rather than as a purely controlled analysis in which the number of students is the only varying factor.

Table 3 reports the mean hypervolume obtained by each algorithm for each instance size, averaging the results over the four instances available for each size group.

Table 3: Mean hypervolume by instance size.

Algorithm	Students				
	200	400	600	800	1000
HH-LAP-SIPA-Lite	0.5942	0.3850	0.7329	0.7453	0.3980
HH-LAP-SIPA-Intensive	0.5083	0.4203	0.3418	0.2755	0.2586
GRASP	0.4437	0.0183	0.0132	0.0101	0.0105
NSGA-III	0.0712	0.0476	0.0285	0.0191	0.0198
Random	0.0264	0.0209	0.0166	0.0137	0.0143

The results show that the proposed HH-LAP-SIPA variants remain the best-performing methods across all instance sizes. The Lite variant obtains the highest mean hypervolume in four of the five size groups, namely 200, 600, 800, and 1000 students, while the Intensive variant obtains the best result for the 400-student instances, with a mean hypervolume of 0.4203. In all size groups, the two proposed methods occupy the first two positions, indicating that their advantage over the baseline methods is maintained as the number of students increases.

The behavior of the baseline methods is more limited. GRASP obtains a competitive value for the smallest instances, reaching a mean hypervolume of 0.4437 for 200 students, but its performance drops sharply from 400 students onward. NSGA-III remains more stable than GRASP for larger instances, but its hypervolume values are consistently much lower than those of the HH-LAP-SIPA variants. The random baseline obtains the lowest or near-lowest values across all size groups, as expected.

The ranking by size confirms this pattern. HH-LAP-SIPA-Lite ranks first in four size groups and second in the remaining one, while HH-LAP-SIPA-Intensive ranks first for 400 students and second for all other sizes. By contrast, the baseline methods remain below the two HH-LAP-SIPA variants across all size groups, with GRASP being competitive only for the smallest instances. This suggests that the proposed hybrid strategies maintain a better relative behavior than the reference methods under the tested computational budget.

However, the lower value obtained by the Lite variant for the 400-student group also shows that performance does not follow a strictly monotonic trend with size. This behavior does not appear to be caused by all 400-student instances performing uniformly worse, but rather by the variability among the generated instances within the same size group. A similar effect can also be observed for the 1000-student group, where the average value of the Lite variant is affected by a subset of more challenging instances. This suggests that the number of students is not the only factor driving the hypervolume values; structural characteristics such as the distribution of companies, dependencies, centers, and supervisors also influence the difficulty of the resulting search landscape.

Figure 3 illustrates the evolution of the mean hypervolume with respect to the number of students.

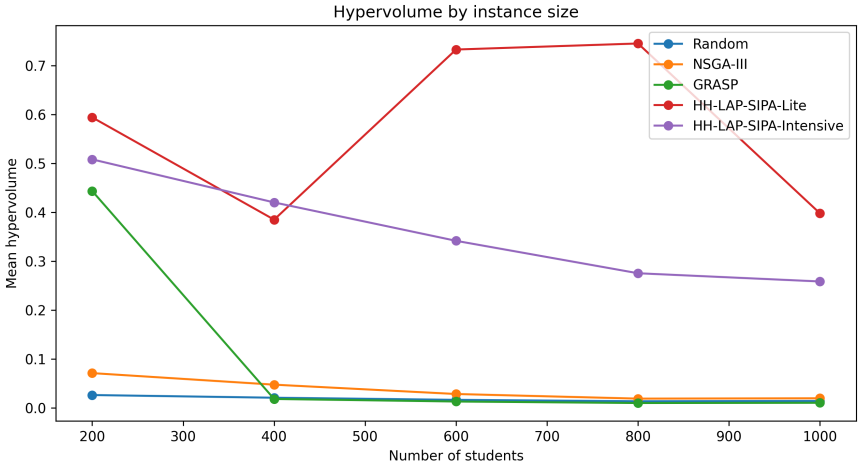


Figure 3: Mean hypervolume by instance size.

As the instance size increases, the search space becomes larger and the interaction between placements, companies, dependencies, and supervision constraints becomes more complex. Under a fixed computational budget, this makes it more difficult for the baseline methods to approximate high-quality Pareto fronts, which helps explain their low hypervolume values for medium and large instances. In contrast, the Lite variant shows the strongest performance for medium and large instances, especially for 600 and 800 students, where it reaches the highest hypervolume values of the whole comparison. This suggests that its hybrid decomposition is able to exploit the structure of larger instances effectively, particularly through the exact assignment component. Although its performance decreases for 1000 students, it still remains clearly above all baseline methods and ahead of the Intensive variant. The Intensive variant follows a more gradual decreasing trend as the instance size increases, while also remaining consistently better than the baselines.

Overall, these results indicate that the proposed HH-LAP-SIPA variants preserve their relative advantage across the tested size groups, while also showing that instance-specific characteristics have a relevant effect on the final hypervolume values.

6.3 Instance-level variability

This subsection analyzes the variability of the hypervolume results at the individual instance level. While the previous subsections showed that the proposed HH-LAP-SIPA variants dominate the baselines globally and across size groups, it is also important to determine whether this behavior is consistent for each particular instance. The analysis therefore focuses on the algorithm obtaining the highest hypervolume in each instance and, in particular, on the differences between the Lite and the Intensive variants.

Table 4 reports the number of instance-level wins obtained by each HH-LAP-SIPA variant for each size group. The results confirm that all instance-level wins are achieved by the proposed variants (HH-LAP-SIPA-Lite obtains the best hypervolume in 15 out of the 20 instances, whereas HH-LAP-SIPA-Intensive obtains the best result in the remaining 5 instances). None of the baseline methods obtains the highest hypervolume in any individual instance.

Table 4: Instance-level wins by size group.

Algorithm	Students					Total
	200	400	600	800	1000	
HH-LAP-SIPA-Lite	3	2	4	4	2	15
HH-LAP-SIPA-Intensive	1	2	0	0	2	5

The distribution of wins shows that the superiority of HH-LAP-SIPA-Lite is generally consistent, but not uniform across all instances. The Lite variant wins all instances with 600 and 800 students, and it also wins three out of the four instances with 200 students. However, HH-LAP-SIPA-Intensive is competitive in some specific cases, particularly in the 400- and 1000-student groups, where both variants obtain two wins each. This indicates that the performance differences are not determined solely by the number of students, but are also influenced by the structural characteristics of each generated instance.

Figure 4 shows the direct hypervolume difference between HH-LAP-SIPA-Lite and HH-LAP-SIPA-Intensive for each instance. Positive values indicate that Lite obtains a higher hypervolume, whereas negative values indicate that Intensive performs better.

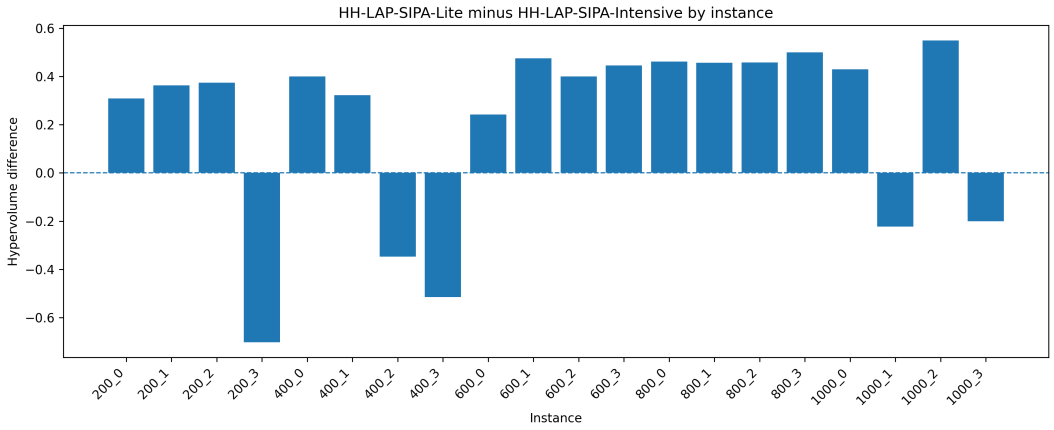


Figure 4: Hypervolume difference between HH-LAP-SIPA-Lite and HH-LAP-SIPA-Intensive by instance.

The figure shows that Lite clearly dominates most instances, with especially large positive differences in the 600- and 800-student groups. In these groups, the Lite variant consistently outperforms Intensive, suggesting that its lighter construction mechanism is particularly effective when combined with the exact assignment component in medium and large instances.

The instances in which HH-LAP-SIPA-Intensive obtains the best result are also informative. Intensive outperforms Lite in one 200-student instance, two 400-student instances, and two 1000-student instances. These cases correspond to the main drops observed for Lite in the size-based analysis, especially in the 400- and 1000-student groups. Therefore, the lower average hypervolume of Lite in those size groups is not due to a uniform degradation across all instances, but rather to a small subset of instances in which the intensive placement–professor construction leads to better Pareto fronts.

Overall, the instance-level analysis reinforces the conclusions obtained from the global comparison. HH-LAP-SIPA-Lite is the most consistently effective variant in terms of number of wins and magnitude of improvement across individual instances, while HH-LAP-SIPA-Intensive remains valuable because it outperforms Lite in a subset of more challenging cases. This suggests that the two variants provide complementary behaviors: Lite is generally more

effective under the fixed computational budget, whereas Intensive can be advantageous when the structure of the instance benefits from a more detailed construction of the supervision layer.

6.4 Per-objective performance analysis

This subsection analyzes the behavior of the compared algorithms with respect to each objective component of the proposed formulation. While the previous subsections used the hypervolume indicator to evaluate the global quality of the approximated Pareto fronts, this analysis examines whether the algorithms differ in the type of objective values they are able to reach. For each approximated Pareto front, the best value reached for each objective was extracted independently. Thus, the reported values should be interpreted as the ability of each algorithm to reach strong objective-specific values within its Pareto fronts, rather than as the performance of a single representative solution.

Table 5 reports, for each algorithm, the average best value reached for each objective over the complete benchmark set. Student utility and company utility are reported per student, since their original aggregate values depend on the number of students in the instance, while supervision dispersion and workload imbalance are reported in the scale used by the evaluator, where lower values indicate better performance. Thus, the first two objectives are maximization components, whereas the last two are minimization components.

Table 5: Per-objective comparison over the complete benchmark set.

Algorithm	Student util.	Company util.	Dispersion	Imbalance	Pareto size
Random	0.1983	0.0876	1.6873	0.0345	66.77
NSGA-III	0.2321	0.1248	1.6864	0.0432	51.10
GRASP	0.2720	0.1513	1.7168	0.0503	3.12
HH-LAP-SIPA-Lite	0.7809	0.5808	1.4307	0.1003	42.08
HH-LAP-SIPA-Intensive	0.7808	0.5806	2.1191	0.0074	110.11

The first two objectives show a clear separation between the proposed HH-LAP-SIPA variants and the baseline methods. Both HH-LAP-SIPA-Lite and HH-LAP-SIPA-Intensive obtain almost identical values for student utility and company utility, with average student utilities around 0.7808 and average company utilities around 0.5807. These values are substantially higher than those obtained by Random, NSGA-III, and GRASP. This indicates that the higher hypervolume values achieved by the proposed variants are strongly associated with their ability to generate assignments with much better student–placement and company-side utility values.

The behavior changes when the supervision-related objectives are considered. For supervision dispersion, lower values are preferred, and HH-LAP-SIPA-Lite obtains the best average result, with a value of 1.4307. The baseline methods obtain intermediate values, whereas HH-LAP-SIPA-Intensive obtains the highest dispersion value. This suggests that the lighter construction strategy is more effective at preserving geographically compact supervision assignments, while the intensive variant tends to explore solutions with larger supervision dispersion.

In contrast, the workload imbalance objective shows the opposite behavior. HH-LAP-SIPA-Intensive achieves the best average imbalance value, with 0.0074, clearly outperforming the rest of the algorithms in this component. Although HH-LAP-SIPA-Lite obtains stronger global hypervolume and supervision dispersion results, it produces a higher imbalance value. This highlights a trade-off between the two variants, since Lite provides better overall Pareto-

front quality and dispersion, whereas Intensive is more effective at balancing the workload among professors.

The average ranking by objective confirms this interpretation. The two HH-LAP-SIPA variants obtain the best ranks for the student and company utility objectives, with only negligible differences between them. HH-LAP-SIPA-Lite obtains the best rank for supervision dispersion, while HH-LAP-SIPA-Intensive obtains the best rank for workload imbalance. Therefore, the two variants are not simply redundant versions of the same search strategy, but rather emphasize different parts of the multi-objective landscape.

The size of the generated Pareto fronts also supports this interpretation. HH-LAP-SIPA-Intensive produces the largest fronts on average, with 110.11 solutions, whereas HH-LAP-SIPA-Lite generates smaller fronts, with 42.08 solutions on average. This suggests that the intensive construction mechanism increases the diversity of the generated fronts, especially in relation to workload balance, although this does not always translate into higher global hypervolume or better geographic compactness. GRASP, in contrast, produces very small fronts on average, which limits its ability to represent meaningful trade-offs among the objectives.

These per-objective results show that the two HH-LAP-SIPA variants reach similar student and company utility values, but differ mainly in the supervision-related objectives, with Lite favoring lower dispersion and Intensive favoring better workload balance and larger Pareto fronts.

6.5 Analysis of HH-LAP-SIPA variants

This subsection compares the two proposed HH-LAP-SIPA variants in order to analyze the effect of the placement–professor construction strategy. HH-LAP-SIPA-Lite uses a lighter construction mechanism, whereas HH-LAP-SIPA-Intensive applies a more detailed construction of the supervision layer. The objective of this analysis is to determine whether the additional intensification introduced by the Intensive variant improves the supervision-related components, or whether the Lite variant is more effective under the fixed computational budget used in the experiments.

Table 6 summarizes the main differences between the two variants. The comparison includes the mean hypervolume, the number of instance-level wins, the two supervision-related objectives, and the average Pareto-front size.

Table 6: Comparison between the two HH-LAP-SIPA variants.

Algorithm	Mean HV	Wins	Dispersion	Imbalance	Pareto size
HH-LAP-SIPA-Lite	0.5711	15	1.4307	0.1003	42.08
HH-LAP-SIPA-Intensive	0.3609	5	2.1191	0.0074	110.11

Since both variants were executed under the same time-budget configuration, their runtime values are practically identical. Therefore, the observed differences should be interpreted as differences in search behavior and objective trade-offs, rather than as a consequence of one variant using substantially more computational time.

Student and company utilities are not included in Table 6 because they are mainly determined by the selected placements and by the exact LAP used for the student–placement assignment layer, which is shared by both HH-LAP-SIPA variants. As observed in the per-objective analysis, both variants obtain almost identical values for these two objectives.

The results show that HH-LAP-SIPA-Lite achieves the best overall performance in terms of hypervolume. It obtains a mean hypervolume of 0.5711 and achieves the best result in 15 out of the 20 instances. HH-LAP-SIPA-Intensive, in contrast, obtains a lower mean

hypervolume of 0.3609 and wins the remaining 5 instances. This indicates that the lighter variant is generally more effective at producing high-quality Pareto fronts under the fixed computational budget.

The supervision objectives reveal a clear trade-off between the two variants. HH-LAP-SIPA-Lite obtains better supervision dispersion, with an average value of 1.4307 compared with 2.1191 for HH-LAP-SIPA-Intensive. This indicates that the lighter construction mechanism tends to produce geographically more compact supervision assignments. However, HH-LAP-SIPA-Intensive obtains a much better workload imbalance value, with 0.0074 compared with 0.1003 for Lite. Thus, the more detailed placement–professor construction is effective at balancing the workload among professors, but this improvement comes at the cost of worse dispersion and lower global hypervolume.

The average Pareto-front size also reflects this difference in behavior. The Intensive variant generates substantially larger fronts, with 110.11 solutions on average, whereas the Lite variant generates 42.08 solutions. This suggests that the intensified construction mechanism explores a broader set of trade-offs, particularly those related to workload balance, thereby producing a greater diversity of nondominated solutions. However, this increased diversity does not translate into better hypervolume. Instead, the Lite variant appears to concentrate the search on regions of the objective space that provide a better overall compromise among the four objectives, resulting in higher-quality Pareto fronts despite containing fewer solutions.

Overall, the comparison shows that the intensified placement–professor construction improves workload balance and increases the size of the generated Pareto fronts, but it does not improve the global hypervolume or the supervision dispersion objective. HH-LAP-SIPA-Lite provides the best overall compromise under the fixed computational budget, whereas HH-LAP-SIPA-Intensive remains useful in specific instances.

7 Conclusions and Future Work

This paper has addressed an extended version of the Student–Industry Placement Allocation problem in which academic supervision is explicitly incorporated into the allocation process. Starting from a competence-based SIPA formulation focused on students and companies, the proposed model introduces academic supervisors, educational centers, supervision capacities, geographical dispersion, and workload balance. This extension makes it possible to evaluate not only whether students and companies are well matched, but also whether the resulting allocation is manageable from an academic supervision perspective.

To solve the resulting multi-objective problem, this paper has proposed HH-LAP-SIPA, a hybrid metaheuristic framework that combines feasible placement selection, placement–professor assignment, and exact Linear Assignment Problems for the student–placement layer. Two variants of the method have been studied. HH-LAP-SIPA-Lite uses a lighter construction mechanism, whereas HH-LAP-SIPA-Intensive applies a more detailed construction of the supervision layer. This design makes it possible to analyze the trade-off between broader exploration and more intensive supervision-oriented construction under a fixed computational budget.

The experimental results show that the proposed HH-LAP-SIPA variants outperform the baseline methods in terms of global Pareto-front quality. In particular, HH-LAP-SIPA-Lite obtains the best overall performance, achieving the highest mean hypervolume and the best result in most of the tested instances. The analysis by instance size and by individual instance confirms that this behavior is not limited to a particular subset of cases, although the results also show that the structural characteristics of each generated instance influence the final

performance.

The per-objective analysis provides further insight into the behavior of the proposed variants. Both HH-LAP-SIPA variants obtain very similar student and company utility values, which reflects the shared use of exact LAPs for the student–placement assignment layer. The main differences appear in the supervision-related objectives. HH-LAP-SIPA-Lite provides better global hypervolume and lower supervision dispersion, whereas HH-LAP-SIPA-Intensive achieves better workload balance and generates larger Pareto fronts. These results show that the academic supervision layer introduces meaningful trade-offs and should not be treated as a secondary post-processing step.

In general, the results support the usefulness of incorporating academic supervision directly into the SIPA formulation and of addressing the resulting problem with a structured, problem-specific metaheuristic. The proposed HH-LAP-SIPA framework exploits the structure of the extended allocation problem by combining placement selection, placement–professor assignment, and exact student–placement optimization. This hybrid decomposition appears to guide the search toward regions of the objective space that provide better trade-offs than those obtained by the baseline methods, while preserving a multi-objective perspective that can provide decision-makers with alternative assignment solutions.

Regarding future work, several lines can be considered. First, the intensive variant could be further improved by designing more effective placement–professor construction and intensification mechanisms, aiming to preserve its advantages in workload balance while reducing its negative impact on supervision dispersion and global hypervolume. Second, additional search operators could be incorporated into HH-LAP-SIPA, including adaptive mechanisms to decide when to favor broader exploration or deeper supervision-layer intensification.

Future work should also evaluate the approach using real-world instances from educational institutions. This would make it possible to assess the practical relevance of the generated Pareto fronts and to study additional constraints that may arise in real internship allocation processes, such as professor availability, temporal scheduling, travel-time restrictions, institutional preferences, or specific supervision policies. Finally, the experimental analysis could be extended with additional multi-objective quality indicators, such as diversity or convergence-related metrics, in order to obtain a more complete characterization of the generated Pareto fronts.

References

[1] M. S. Yassen, V. Sánchez-Anguix, J. M. Alberola, and F. Villa, "An integer linear programming model based on competences for student-industry placement allocation," in *Proceedings of the 39th Annual ACM Symposium on Applied Computing*, ser. SAC '24. Association for Computing Machinery, 2024, pp. 104–105.

[2] D. Manlove, D. Milne, and S. Olaosebikan, "An integer programming approach to the student-project allocation problem with preferences over projects," in *Combinatorial Optimization*. Springer, 2018, pp. 313–325.

[3] M. Chiarandini, R. Fagerberg, and S. Gualandi, "Handling preferences in student-project allocation," *Annals of Operations Research*, vol. 275, no. 1, pp. 39–78, 2019.

[4] G. Candel, V. Sánchez-Anguix, J. M. Alberola, V. Julián, and V. Botti, "An evolutionary metaheuristic for forming teams in the classroom with constraints," *Neurocomputing*, vol. 638, p. 130138, 2025.

[5] V. Sánchez-Anguix, R. Chalumuri, R. Aydoğan, and V. Julián, "A near pareto optimal approach to student–supervisor allocation with two sided preferences and workload balance," *Applied Soft Computing*, vol. 76, pp. 1–15, 2019.

[6] A. Y. Ali, "Greedy algorithm for solving student allocation problem in internship program: A case study," *International Journal of Research in Industrial Engineering*, vol. 10, no. 2, pp. 155–164, 2021.

[7] D. Thiruvady, K. Morgan, S. Bedingfield, and A. Nazari, "Allocating students to industry placements using integer programming and ant colony optimisation," *Algorithms*, vol. 14, no. 8, p. 219, 2021.

[8] M. S. Yassen, V. Sánchez-Anguix, J. M. Alberola, F. Villa, V. Julián, and V. Botti, "Exploring an artificial intelligence model based on integer linear programming for allocating students to industry placements," in *Methodologies and Intelligent Systems for Technology Enhanced Learning, 14th International Conference*, ser. Lecture Notes in Networks and Systems, vol. 1171. Springer, 2024, pp. 90–102.

[9] C. A. Coello Coello, G. B. Lamont, and D. A. Van Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2nd ed. New York, NY: Springer, 2007.

[10] R. E. Burkard, M. Dell'Amico, and S. Martello, *Assignment Problems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2009.

[11] R. Jonker and A. Volgenant, "A shortest augmenting path algorithm for dense and sparse linear assignment problems," *Computing*, vol. 38, no. 4, pp. 325–340, 1987.

[12] F. Glover, M. Laguna, and R. Martí, "Fundamentals of scatter search and path relinking," *Control and Cybernetics*, vol. 29, no. 3, pp. 653–684, 2000.

[13] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part i: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.

[14] T. A. Feo and M. G. C. Resende, "Greedy randomized adaptive search procedures," *Journal of Global Optimization*, vol. 6, no. 2, pp. 109–133, 1995.

[15] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength pareto approach," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 4, pp. 257–271, 1999.

[16] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.